

Literature Review

Aayuman Ghangurde
Purdue University

Sai Palaparthi
Purdue University

Rakib Ahsan
Purdue University

Zeke Ulrich
Purdue University

1 Related Work

1.1 Directly Relevant Papers

Storing passkeys (device-bound WebAuthn/FIDO2 private keys) inside a TPM on Linux sits at the intersection of hardware-rooted key storage, credential portability/backup, and the broader passkey ecosystem. The five papers below, all published in 2025 or later, are the ones most directly relevant to this problem; together they also outline the gap that our work aims to fill: none of them implements or evaluates a Linux-native TPM 2.0 backend that manages PCR-policy sealing, key-slot capacity, and recoverable export/import for WebAuthn credentials.

QES-Backed Virtual FIDO2 Authenticators [?]. Bicakci et al. note that FIDO2/WebAuthn traditionally relies on device-bound cryptographic keys that are not naturally portable across user devices, and that current passkey deployments address this by synchronizing credentials through platform-specific cloud ecosystems – requiring users to trust the corresponding cloud or phone provider with their authentication material. As an alternative, they bridge qualified-electronic-signature (QES) smart-card/PKCS#11 modules, which provide high-assurance, hardware-rooted identity but are not directly compatible with WebAuthn, by wrapping a QES-grade PKCS#11 key inside a virtual FIDO2 authenticator and encrypting the private key material for cloud synchronization. They implement a baseline architecture in which the cloud stores only ciphertext while decryption capability stays anchored in the user’s hardware token, and they design (but do not implement) a hardened, OPRF-based variant that binds synchronization keys to a local user-verification factor so they cannot be repurposed outside FIDO2 semantics. **Shortcoming:** the hardened variant is analyzed only on paper, and the “hardware token” in both designs is a QES smart card/PKCS#11 module rather than a general-purpose TPM – there is no treatment of TPM-native key hierarchies, PCR policies, or the Linux platform stack, which is exactly the

backend our work targets.

Vaulted Passkeys [?]. This paper starts from the observation that hardware authenticators deliberately resist private-key extraction, yet replacement, disaster recovery, and controlled migration create a legitimate need for credential portability. Existing guidance (registering a second authenticator in advance) is redundancy rather than a true backup of the original credential. The authors propose *Vaulted Passkeys*, a device-bound architecture that exports protected credential state while the source authenticator is still available and restores it to hardware acquired later, without cloning a full authenticator or exposing plaintext private keys to routine desktop software; a random 256-bit vault key protects authenticated credential envelopes through HKDF-separated keys and four explicit AEAD profiles. **Shortcoming:** the design is authenticator-agnostic and does not engage with the specifics of TPM 2.0 (e.g., sealing envelopes to PCR state, using the TPM’s own key-wrapping hierarchy instead of an external vault key), so it does not show how such export/import would actually be realized – or how safe it would be – for a TPM-backed Linux passkey store.

An Analysis of Attack Vectors Against FIDO2 Authentication [?]. Motivated by the continued prevalence of phishing (the Anti-Phishing Working Group reported over 890,000 attacks in Q3 2025 alone), Berladskeyy and Aßmuth examine passkeys/FIDO2, where a private key is stored on the user’s authenticator device and the server holds only the corresponding public key; authentication happens locally on the device (e.g., via fingerprint or PIN) rather than through server-side hash comparison, and attestation lets a relying party cryptographically verify that a key was generated by a specific authenticator, such as a hardware security key. **Shortcoming:** the analysis treats the authenticator largely as a black box and focuses on protocol- and phishing-level attack vectors; it does not analyze platform-specific weaknesses of a particular secure element (e.g., misconfigured PCR policies

or TPM sealing on Linux), leaving open exactly the kind of implementation-level threat model our work needs to address.

Verifiable Passkey [?]. Mitra and Sethuraman point out that secure storage modules such as a TPM or a physical security key have limited storage space, which caps the number of passkeys a user can register – for example, a Yubikey 5 with older firmware stores only 25 passkeys, while newer firmware allows up to 100 – and that the common workaround, federated single sign-on, creates a privacy risk because the identity provider can track a user across the services it authenticates them to. They propose *Verifiable Passkey*, a standard that lets a user reuse a single passkey created for a Verifiable-Credential issuer across platforms without exposing that cross-service tracking surface. **Shortcoming:** this is the one paper in our set that names TPM storage-capacity limits explicitly as a motivating problem, but its fix works around the constraint at the credential-issuer/protocol layer rather than addressing efficient, scalable on-TPM key storage itself – e.g., through key derivation or wrapping schemes that let many credentials share a small number of resident TPM key slots, which is the angle our work takes.

The Passwordless Authentication with Passkey Technology from an Implementation Perspective [?]. Tran et al. highlight the key techniques involved in implementing a passkey-based authentication system and the considerations needed when integrating its components during system development, so that users can securely access their accounts with minimal complexity while the system still balances security, usability, and performance; they compare passkeys against TOTP and evaluate both mechanisms’ feasibility in real-world deployments. **Shortcoming:** the paper is written from an application/backend-integration perspective (rate limiting, OAuth2, two-factor flows) rather than the client-side secure-storage layer, and it does not discuss how private key material is actually sealed to platform hardware such as a TPM or Secure Enclave – precisely the layer our work is concerned with.

1.2 Last Paper (Not Directly Relevant) – Need Ten

Citation: [1].

References

- [1] Carl A. Waldspurger. Memory resource management in VMware ESX server. In *USENIX Symposium on Operating System Design and Implementation (OSDI)*, pages 181–194, 2002. <https://www.usenix.org/legacy/event/osdi02/tech/waldspurger/waldspurger.pdf>.